

Software Requirements Specification

Equity Backtesting Engine

Version	Date	Author	Change Description
1.3	2025-11-15	Technical Architect	Added architectural clarifications for MVP development
1.4	2025-05-12		Clarification rechecked
1.5	2025-12-10	Business Analyst	Integrated Final Entry/Exit Rules; Resolved Risk-Free Rate & Lookback clarifications
1.6	2025-12-10	Business Analyst	Revised Purpose to specify 13F filtration and walk-forward validation methodology
1.7	2025-12-10	Business Analyst	Refined Definitions: Strategy Heartbeat, Universe (PIT), and Sub-universe (Eligible Targets)
1.8	2025-12-10	Business Analyst	Adopted Event-Condition-Action (ECA) taxonomy in Functional Requirements (Section 3.1)
1.9	2025-12-10	Business Analyst	Removed FR-3.1.C.6 (User-defined risk

			management rules)
1.10	2025-12-10	Business Analyst	Updated FR-3.1.B.2 to mandate BigQuery storage for SEC filings via APIs
1.11	2025-12-10	Business Analyst	Expanded FR-3.1.B.3 (Universe Filtration) and detailed FR-3.1.B.4 (Data Validation)
1.12	2025-12-10	Business Analyst	Mandated Full History Retention in FR-3.1.B.2 and clarified Lookback vs Retention in FR-3.1.C.1/2
1.13	2025-12-10	Business Analyst	Overhauled FR-3.1.C.11 to implement Mandatory Thesis Drift, Insider Reversal, and updated Trailing Stop/Dead Money logic
1.14	2025-12-10	Business Analyst	Added FR-3.1.C.13 for Conviction Ranking, Replacement Protocol, and Cash Drag Management; Updated FR-3.1.C.4 constraints
1.15	2025-12-10	Business Analyst	Expanded Section 5 with detailed System Module

			descriptions and Google Cloud Implementation specifics
1.16	2025-12-10	Business Analyst	Replaced Appendix A with Technical Architecture & Backtesting Library Recommendation (LEAN)
1.17	2025-12-10	Business Analyst	Updated Section 2.4 to explicitly allow purchasing paid API keys/licenses for data providers
1.18	2025-12-10	Business Analyst	Detailed FR-3.1.E.5 and added Appendix C to specify required Advanced Visualizations (Monte Carlo, Heatmap, WFE, Stress Test)
1.19	2025-12-10	Business Analyst	Renumbered Strategy Configuration section to ensure sequential ordering (C.1 to C.12) and updated cross-references
1.20	2025-12-11	Business Analyst	Updated FR-3.1.C.9 to include specific Primary Entry Signals: Doubling Down, Insider Buying, and

			Institutional Herding
1.21	2025-12-11	Business Analyst	Refined FR-3.1.C.9 Logic Flow: Explicitly defined Sequence of Signal Generation (A, B, C) -> Technical Confirmation

1. Introduction

1.1 Purpose

The purpose of the project is to carry out backtesting of entry, exit, and position sizing signals that are generated after filtration of the universe of 13F filings. The backtesting engine will carry out backtesting in walk-forward (train-test mode) to generate robust validation via parameter sensitivity analysis, Monte Carlo simulation, and stress testing periods analysis.

This document specifies the software requirements for the Equity Backtesting Engine. It is intended for all stakeholders, including project managers, developers, QA testers, and product owners. The primary technical audience is software developers with expertise in Python.

1.2 Scope

The system will allow users to define, test, and analyze equity trading strategies against historical market data. It will simulate trade execution based on user-defined logic and provide detailed performance metrics. Inputs can be provided via a simple UI or spreadsheet.

This SRS covers:

- **Data Ingestion:** SEC filings (13F/D/G, Form 4) and on-demand market data retrieval.
- **Strategy Configuration:** User-defined trading strategies including:
 - Clear entry rules (derived from SEC filings).
 - Position sizing and allocation rules.
 - Exit rules.
 - Risk management and rebalancing rules.
- **Backtest Execution:** Logic for chronological simulation.
- **Performance Reporting:** Comprehensive analysis and metrics.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
------	------------

Backtest	A simulation of a trading strategy using historical data to assess potential profitability.
Entry Signal	A condition or set of conditions that trigger a simulated "buy" order.
Exit Signal	A condition or set of conditions that trigger a simulated "sell" order.
P&L	Profit and Loss.
Drawdown	The peak-to-trough decline in the value of a portfolio.
SRS	Software Requirements Specification.
PIT	Point-in-Time (Data availability based on publication timestamp, not reporting period).
Strategy Heartbeat	The periodic trigger (e.g., daily, monthly, or quarterly) that initiates the re-scanning of the universe for new signals.
Universe	The initial search space at a specific Point-in-Time (t), comprising the set of all tickers with available data (e.g., S&P 1500 constituents). It is not a static list but the dynamic output of a filter function applied to the total available market data to ensure PIT accuracy.
Sub-universe	A computation that runs at specific intervals (T) to generate a list of Eligible Targets based on strategy criteria. Like the Universe, it is not a static list but the output of a filter function derived from the Universe.

2. Overall Description

2.1 Product Perspective

The Equity Backtesting Engine will be a core component of our analytical toolkit. It will operate on a repository of historical market data and receive strategy definitions from users (via UI or

API). It will produce detailed reports as its primary output.

2.2 Product Functions

- **Data Management:** Retrieve historical market data (OHLCV, dividends, splits) on-demand via API. Ingest and store SEC filings and index data.
- **Strategy Configuration:** Allow users to define strategy parameters, including entry/exit rules, order sizing, and commission models.
- **Backtest Execution:** Process historical data chronologically, applying strategy rules, simulating trades, and tracking a virtual portfolio.
- **Performance Analysis:** Calculate and present comprehensive performance metrics (Sharpe, Sortino, Alpha, Beta, Max Drawdown) and visualizations.

2.3 User Characteristics

- **Quantitative Analysts (Quants):** Primary users. They require detailed, accurate reporting but a simple configuration interface to validate judgments.
- **Portfolio Managers:** Use the system to validate strategy ideas and understand risk profiles.
- **Developers:** Integrate with the engine via APIs to build custom workflows.

2.4 Constraints

- **Data Providers:** Must use approved providers.
 - **Market Data:** Alphavantage.co (Free Tier initially) for OHLCV and corporate actions.
Note: Data is retrieved on-demand and not stored.
 - **SEC Data:** SEC-API.io or Whalewisdom.com.
 - **Paid Access Provision:** While the system is designed to respect free tier limits where possible, **it is explicitly permitted to purchase API calls, premium subscriptions, or license keys** (e.g., AlphaVantage Premium, SEC-API Professional) if necessary to meet data volume, rate-limit, or reliability requirements.
- **Development Standards:** Must follow CI/CD and coding standards compatible with AI-assisted tools (Cursor, GitHub Copilot).
- **Mock Data:** Given the limits of free APIs during development, the MVP **must** include a "Mock Data Provider" (local CSV generator with randomized noise) for development and CI/CD pipelines.
- **Technology Stack:**
 - Language: **Python**.
 - Data Warehouse: **Google BigQuery** (for SEC filings and index data).

2.5 Assumptions and Dependencies

- Historical market data is assumed to be available and accurate.
- Price data from the API is adjusted for splits, bonuses, and rights issues.
- Dividends are assumed to be reinvested.
- Users have a foundational understanding of trading concepts.

3. Specific Requirements

3.1 Functional Requirements

The system must implement a core workflow guiding the user through a multi-step process:

Universe → Sub-Universe → Entry/Exit Application.

To support this workflow efficiently and deterministically, the system must adopt an **Event-Condition-Action (ECA) taxonomy**:

- **Event:** OnSecFilingReceived(filing)
 - Triggered when a new 13F/D/G or Form 4 filing is ingested.
- **Condition:** if (filing.Fund.AUM > 1B and stock.MktCap > 3B)
 - Represents the application of Universe and Sub-universe filters (FR-3.1.C.3) to validate the signal source and target asset.
- **Action:** EvaluateTechnicalSignal() and GeneratePortfolio()
 - **Action 1:** Check technical entry rules (**FR-3.1.C.9**) for the qualified stock.
 - **Action 2:** If confirmed, allocate to the portfolio based on sizing rules (**FR-3.1.C.4**) and Ranking Logic (**FR-3.1.C.10**).

3.1.A Core Strategy Workflow

FR-3.1.A.1: The system must provide a user workflow following these stages:

1. **Strategy Setup:** Initial configuration.
2. **Stock Selection Model:** Define screen for entry generation.
3. **Entry and Position Sizing:** Finetune model and assign weights.
4. **Entry Scheduling:** Define frequency.
5. **Exit Model:** Define exit signals.
6. **Risk Management:** Define risk rules.
7. **Parameters:** Define benchmarks/others.
8. **Backtest:** Execute and review.

3.1.B Data Management

FR-3.1.B.1: The system must retrieve historical daily EOD OHLCV data from the Alphavantage.co free tier API on-demand. Data is assumed to be adjusted for corporate actions.

FR-3.1.B.2: Data of SEC 13F, 13D/13G, and Form 4 filings are to be extracted and stored in Google BigQuery using SEC APIs / python library.

- **Data Retention Policy:** The system must enforce **Full History Retention** for all ingested SEC filings. Storage costs in BigQuery are negligible compared to the analytical value of historical signal data. No historical data should be purged or limited by arbitrary windows (e.g., 9 quarters).
- **PIT Accuracy:** Use "Filing Date" as the availability date. No signals generally before Filing Date.

- **Mapping:** Use OpenFIGI or SEC-API.io for CUSIP <> Ticker mapping.

FR-3.1.B.3: The system must expand Universe & Sub-Universe filtration capabilities to support criteria based on all available fields in 13F/D/G and Form 4 filings, in addition to the base Market Cap > \$500mn rule. The system must support filtering on:

- **Tickers & Identifiers:** CUSIP, Ticker Symbol (mapped), and Issuer Name.
- **Institutional Investor:** Fund Name, Manager Name, CIK, and report type (13F-HR, 13G, etc.).
- **Transaction Filters (13F):** Share count, Value (x\$1000), Investment Discretion (Sole/Shared), Voting Authority (Sole/Shared/None).
- **Insider Buying/Selling Parameters (Form 4):** Relation of Reporting Person (Director, Officer, 10% Owner), Transaction Code (e.g., P - Purchase, S - Sale), Transaction Date, Deemed Execution Date, and Amount of Securities Acquired/Disposed.

FR-3.1.B.4: The system must perform the following validation checks on SEC filings data to ensure integrity before use:

- **Ticker/CUSIP Validation:** Reject or flag records where CUSIP cannot be mapped to a valid Ticker or where the Ticker was not active on the filing date.
- **Institutional Investor Validation:** Verify that the filingManager (Institutional Investor) name is not null and matches known formatting rules. Ensure consistent CIK mapping.
- **Transaction Filter Validation:**
 - Ensure shares and value fields are non-negative.
 - Flag potential data errors where the implied price (value / shares) deviates by >50% from the market close price on the Report Period End Date.
- **Insider Parameter Validation (Form 4):**
 - Ensure transactionCode is a valid SEC code (e.g., 'P', 'S', 'A', 'D').
 - Verify transactionDate is not in the future relative to the Filing Date.
 - Validate that the Reporting Owner has at least one defined relationship (Director, Officer, 10% Owner, or Other).

3.1.C Strategy Configuration

FR-3.1.C.1: User must be able to define a backtest period (start/end date) supporting a 10+ year horizon. **Clarification:** This backtest period utilizes the full depth of the retained data. It is distinct from specific "Lookback Windows" (e.g., evaluating growth over the last 9 quarters) which are strategy parameters, not data storage limits.

FR-3.1.C.2 Universe: A universe of equities is automatically created based on the **full Point-in-Time (PIT) history** available in BigQuery. The system must allow the user to apply a specific "Lookback Window" (Default: 9 Quarters) as a filter to identify active or relevant signals, but this must not constrain the underlying data availability for the backtest.

FR-3.1.C.3 Sub-universe Creation: The system must implement a filtering process to narrow the Universe into a qualified "Sub-universe."

- **Criteria:** Users can combine criteria using logical operators.

- **Investor Qualification (Fund Characteristics):**
 - Min AUM: > \$1B.
 - Track Record: ≥ 8 consecutive quarters.
 - Concentration: No single position > 30-40% of portfolio.
 - Turnover: < 40% (Formula: $(\text{Min}(\text{Purchases}, \text{Sales}) * 2) / \text{Avg Portfolio Value}$). **Note:** "Avg Portfolio Value" must be calculated using a **4-quarter rolling average**.
 - Investor Type: Exclude Index Funds, Passive ETFs, Banks/Brokers.
 - Holdings Count: < 40 stocks.
 - Top 10 Holdings Value: > 30% of total primary quarter AUM.
- **Investee Company Qualifications (Stock Characteristics):**
 - Market Cap: > \$3B.
 - Index Membership: S&P 1500.
- **Transaction Filters:**
 - Min Buy Value: > \$10M in primary and secondary quarters.
 - Share count increase > 5%.
 - Combined 2-quarter purchase ≥ 2% of average AUM.
- **Insider Qualification (Form 4):**
 - Roles: CEO, CFO, COO, President, Chairman.
 - Exclude: 10b5-1 plans (fixed calendar).
 - Value: ≥ Max(\$100k or 0.05% Market Cap).
 - Volume: Total Bought/Sold (90d) ≥ 3.
 - Cluster: Unique insiders ≥ 2 & ≥ \$200k aggregate within 30 days.
 - Timing: Insider buy must fall within Quarter End + 60 days where hedge fund doubled stake.

FR-3.1.C.4 Position Sizing & Constraints:

- **Sizing:** Static position size of 5% of total portfolio value.
- **Constraints:**
 - **Min 5 stocks** to be active (See **FR-3.1.C.10.3** for Cash Drag Management).
 - **Max 20 stocks.** If >20 candidates qualify, the system will select the top 20 based on the **Conviction Ranking Algorithm (FR-3.1.C.10.1)**.

FR-3.1.C.5: The system must implement user-defined exit rules. Users must be able to select one or a combination of pre-defined exit logic modules (Detailed in **FR-3.1.C.11**). Any risk management trigger (stop loss or profit booking) must be calculated based on the 'close' of the daily candle unless otherwise specified in the specific rule logic.

FR-3.1.C.6: The system must implement a **heartbeat mechanism** that results in signal generation from the sub-universe. At each heartbeat period, the "Sub-universe Creation" logic (Section 4 of FR-3.1.C.3) will be re-run to generate the new list of qualified assets. At each month, sections 1, 2, and 3 of FR-3.1.C.3 will run to generate the universe.

FR-3.1.C.7: The Backtesting inputs must be able to model transaction costs, including commissions and slippage.

- **Commission Default:** \$0.005 per share.

- **Slippage Default:** 25 bps of execution price.

FR-3.1.C.8: The user must be able to compare a performance outcome with the benchmark INDEXSP: SP500TR.

FR-3.1.C.9: Entry Logic: Signals & Confirmation

The system must generate trade entries based on a specific, sequential logic flow. The process must first generate all primary signals before applying technical filters.

Step 1: Primary Signal Generation

The system shall generate all 3 of the following primary signals for the entire Sub-universe at the strategy heartbeat. This step ensures that a complete set of potential candidates is identified based on smart money behavior.

- **Signal A: Doubling Down Signal**
 - **Logic:** Triggered if Stock Price < Investor's Estimated Cost Basis **AND** Share Count Increased.
- **Signal B: Insider Buying Signal**
 - **Logic:** Triggered if Large Inst Buy **AND** Insider Buy > 10% Increase.
- **Signal C: Institutional Herding Signal**
 - **Logic:** Triggered if 1 Large Inst Buy **AND** 2+ Large Inst Followers.

Step 2: Signal Aggregation & Candidate Selection

Stocks that have triggered ANY of the three primary signals (A, B, or C) are aggregated into a "Primary Candidate List".

Step 3: Technical Confirmation (FR-3.1.C.9.2)

The system must then apply the following technical filters only to the stocks in the Primary Candidate List. A trade is executed if and only if a candidate meets ALL of these technical conditions on the signal date or subsequent days:

1. **Price Breakout:** Close > 10-day High.
2. **Trend Filter:** Close > 50-day Simple Moving Average (SMA).
3. **Momentum Filter:** Relative Strength Index (RSI 14-period) > 45.

FR-3.1.C.10: Portfolio Construction & Ranking Logic

The Entry Logic serves as the bridge between the Candidate Pipeline (Sub-universe) and the Order Management System. It must handle the ranking of candidates when the number of eligible stocks exceeds the portfolio slots.

- **FR-3.1.C.10.1: Conviction Ranking Algorithm**

To deterministically select the top 20 candidates (per FR-3.1.C.4), the system shall calculate a Conviction Score for every eligible stock at each rebalancing event.

Composite Score Formula:

$$S_{\text{conviction}} = w_1(I_{\text{herding}}) + w_2(I_{\text{insider}})$$

- I_{herding} (**Institutional Herding Score**): Normalized score (0-100) based on the net increase in shares held by the qualified funds (from 13F).
- I_{insider} (**Insider Confidence Score**): Normalized score (0-100) based on the aggregate dollar value of open-market purchases by "C-Level" insiders in the last 90 days (from Form 4).
- **Weights:** User-configurable weights (Default: $w_1=0.6$, $w_2=0.4$).

Tie-Breaking Logic: If $S_{\text{conviction}}$ is identical for two candidates, the system prioritizes the stock with the Lower Market Capitalization (implying higher potential beta/growth).

- FR-3.1.C.10.2: Replacement Protocol (Portfolio Churn Buffer)

To prevent excessive turnover (e.g., buying Stock #20 and selling Stock #21 for a marginal gain), the system must implement a Rank Buffer.

- Logic: A new candidate (C_{new}) enters the portfolio only if:
 $R_{\text{Rank}}(C_{\text{new}}) < R_{\text{Rank}}(H_{\text{worst}}) - B$

(Where B is the Buffer, Default: 5 spots).

- **Example:** To replace the 20th ranked holding, a new candidate must be ranked #15 or better.

- FR-3.1.C.10.3: Cash Drag Management

To prevent concentrating risk in too few names, the system must inhibit entries if the pipeline is weak.

- **Logic:** If $\text{Count}(\text{Qualified Candidates}) < 5$: **Portfolio = 100% Cash.**
- **Rationale:** A universe yielding only 1-4 signals indicates a "risk-off" or "data error" environment; the strategy relies on diversification.

FR-3.1.C.11: Exit Rules (Exit Modules)

The system must provide the following four distinct exit modules based on Thesis Drift and Risk Management. The system must exit a position if the underlying investment thesis is invalidated.

- **Exit Module 1: Mandatory Thesis Drift Exit (Smart Money Follow-through)**

- **Requirement:** 13F/Fundamental Invalidation. If the smart money leaves, the trade must close.
- **Trigger Event:** Processing of a new 13F batch (Quarterly).
- **Exit Condition:** Trigger exit if **ANY** of the following occur:
 - The stock is no longer held by **any** of the "Qualified Funds" (defined in Sub-universe).
 - The "Net Institutional Ownership" drops by $> 20\%$ Quarter-over-Quarter.
 - The stock fails the fundamental filters (e.g., Market Cap drops below \$3B).
- **Execution:** Market Order at Open ($T+1$) following the Signal Date (T = Filing Date).

- **Exit Module 2: Insider Reversal Exit (Form 4 Selling Spikes)**

- **Requirement:** Significant Insider Selling. While insiders sell for many reasons, a massive cluster is a negative signal.
- **Trigger Event:** Processing of Form 4 filings.
- **Exit Condition:** Trigger exit if Aggregate Insider Sales (Form 4) in the last 30 days $> \$5,000,000$ **AND** $> 50\%$ of total insider holdings.

- **Exit Module 3: Stop-Loss and Take-Profit (Intra-Quarter)**

- **Requirement:** High-Water Mark Trailing Stop to protect the portfolio between quarterly 13F updates.
- **Logic:** $P_{\text{exit}} = \text{Max}(\text{Price}_{\text{history}}) \times (1 - \text{Trailing_Stop_Pct})$
- **Parameter:** Trailing_Stop_Pct (e.g., 15%).
- **Trigger:** If $C_t < P_{\text{exit}}$, generate Sell Signal.
- **Granularity:** Checked Daily on Close. Executed Next Open.

- **Exit Module 4: Time-Based Stale Exit (Dead Money Rule)**
 - **Requirement:** Dead Money Rule to free up capital.
 - **Exit Condition:** If a position has been held for **> 4 Quarters AND** Total Return < 0%.
 - **Action:** Force Exit.

FR-3.1.C.12: Default Recommended Strategy Configuration

To assist new users (per NFR-4.4.1), the system must default to the "Combined Professional" configuration, which enables the following simultaneous rules:

1. **Entry:** All conditions in **FR-3.1.C.9**.
2. **Exit:** Enable **Module 1** (Thesis Drift), **Module 3** (Trailing Stop), and **Module 4** (Time-Based/Dead Money).

3.1.D Backtest Execution

FR-3.1.D.1: The engine must use a sophisticated, event-driven backtesting methodology.

FR-3.1.D.2: The engine must apply strategy logic at each bar to check for signals.

FR-3.1.D.3: Execution Logic:

- **Standard:** All transactions (buys, sells, stops) are filled at the 'open' price of the *next* trading day (T+1) to avoid look-ahead bias.
- **Delisting/Forced Exit:** If a stock is delisted, execution occurs at the last available "Close" price on the date of delisting.

FR-3.1.D.4: The engine must maintain a virtual portfolio supporting fractional share positions.

FR-3.1.D.5: Prevent look-ahead bias (only T data used at time T) and survivorship bias.

3.1.E Results and Reporting

FR-3.1.E.1: The system must generate a summary report with key performance metrics.

- **Performance:** Cumulative Return, CAGR, Volatility, Max Drawdown, RoMaD.
- **Metrics vs Benchmark:** Beta, Alpha, Sharpe Ratio, Sortino Ratio, Information Ratio.
- **Risk-Free Rate Definition:** For Sharpe/Sortino calculations, the system must use a **dynamic time-series of the 3-Month Treasury Bill Yield (TB3M)**.
- **Risk:** VaR, Expected Shortfall (cVaR).
- **Statistics:** Best/Worst Day/Month/Year, Win Rates (Day/Month/Year).

FR-3.1.E.2: Time-series chart of portfolio equity curve vs. benchmark.

FR-3.1.E.3: Detailed trade log.

FR-3.1.E.4: Trade Summary spreadsheet (wins, losses, costs).

FR-3.1.E.5: The system must generate the following four Advanced Visualizations to validate strategy robustness (See Appendix C for detailed examples):

1. **Monte Carlo Simulation Cone:** Probability cone showing 95% and 99% confidence intervals for equity growth.
2. **Parameter Sensitivity Heatmap:** Matrix visualizing stability across varying parameters (e.g., Moving Average period vs. Stop Loss size).

3. **Walk-Forward Cluster Matrix:** Color-coded matrix showing Out-of-Sample efficiency across re-optimization periods.
4. **Stress Testing Bar Chart:** Comparison of strategy drawdown vs. benchmark drawdown during specific historical crash events (e.g., Dot Com, 2008 Crisis).

4. Non-Functional Requirements (NFRs)

4.1 Scalability

NFR-4.1.1: Scale horizontally to handle increasing entry signal volume.

NFR-4.1.2: Support parallel backtests (perpetual runs).

NFR-4.1.3: BigQuery must scale for growing SEC data.

4.2 Reliability

NFR-4.2.1: Deterministic results (same inputs = same outputs).

NFR-4.2.2: Robust error handling.

4.3 Usability

NFR-4.3.1: "Super simple" interface. Setup time < 15 minutes.

NFR-4.3.2: APIs documented with OpenAPI/Swagger.

4.4 Security

NFR-4.4.1: Strategy privacy enforcement.

NFR-4.4.2: Adherence to company security policies.

5. System Modules

Modularization is the process of subdividing the system into smaller, independent units (modules) that function together to deliver the complete application. This approach reduces complexity by separating concerns (e.g., isolating data fetching from strategy logic), enables parallel development by different team members, and ensures that individual components can be tested, optimized, or replaced without disrupting the entire system.

Based on the requirements, the system will be constructed using the following core modules:

5.1 Data Ingestion & Storage Module

The Data Ingestion & Storage Module forms the foundational layer of the system and is responsible for acquiring, validating, standardizing, and storing all external datasets required by the platform. This includes 13F filings from SEC EDGAR, insider Form 4 transactions, historical price and corporate action datasets, fundamental datasets such as market capitalization or valuation ratios, and optional factor datasets that enable more advanced analytics. Once ingested, the module normalizes structures, maps CUSIP identifiers to tickers and historical identifiers, resolves amendments, and consolidates all information into a unified and queryable data warehouse.

This module's role in the system is critical because every subsequent component—signal

generation, backtesting, analytics, and the strategy builder—depends on the correctness, completeness, and temporal accuracy of this data. Errors or gaps in ingestion directly compromise the reliability of strategy outcomes. Ultimately, this module delivers clean, time-aligned datasets stored in analytical-grade tables, enabling consistent retrieval and reproducibility of signals and backtests.

The module interacts primarily as an upstream data provider to every other component of the platform. The Signal Engine draws on its holdings, insider, and factor tables to generate institutional and insider signals. The Backtest Engine uses adjusted historical prices, corporate actions, and fundamentals for trade simulation and risk modeling. The Strategy Builder and Analytics module also retrieve various metadata including lists of funds, securities, sectors, and factor fields.

Google Cloud Implementation:

On Google Cloud, this module naturally leverages BigQuery for storing structured historical data, Cloud Storage for raw files, Cloud Run or Cloud Functions for ingestion pipelines, Cloud Scheduler for recurring data refreshes, Pub/Sub for asynchronous processing, MemoryStore for low-latency key-value lookups (such as CUSIP mappings), and Secret Manager for securely storing API keys or EDGAR credentials.

5.2 Signal Engine Module

The Signal Engine is the analytical heart of the platform. Its purpose is to transform raw historical data into actionable buy and sell signals that reflect institutional activity, insider trading conviction, factor alignment, and optional technical filters. It computes a series of advanced metrics—including herding strength indices, net institutional demand, conviction levels, multi-fund confirmations, insider accumulation intensity, and composite scores that combine institutional signals with price or fundamental filters. The module applies user-defined strategy rules to determine whether each stock at each point in time qualifies as a valid trade signal.

Within the overall system, this module plays the role of the “decision-maker.” Whereas the Backtest Engine is execution-oriented, the Signal Engine determines what should be purchased, why, and when, based on all available smart-money and fundamental information. If this module is inaccurate, the simulation results—even if mechanically correct—fail to reflect genuine institutional behavior or insider conviction.

The module outputs structured records of signals, each containing the stock, date, signal type, conviction score, reasoning metadata, and any underlying measures used in decision-making. These outputs are consumed directly by the Backtest Engine during portfolio construction and execution simulation. Additionally, the Analytics module uses signal-level metadata to compute attribution analyses, conviction breakdowns, and fund-level performance contributions. The Strategy Builder also depends on this module to preview how many signals a given rule set would historically produce.

Google Cloud Implementation:

On the Google Cloud platform, the Signal Engine is most effectively deployed on Cloud Run as

a Python-based microservice, with intermediate caching via MemoryStore to reduce computation time. BigQuery stores computed signal tables and intermediate transformations, while Cloud Composer (Airflow) or Cloud Run jobs orchestrate scheduled recomputation or batch runs. If needed, Vertex AI Feature Store can be used for maintaining historical features used repeatedly across simulations.

5.3 Backtest Engine Module

The Backtest Engine serves as the execution and simulation engine for all strategies created within the platform. Once it receives signals from the Signal Engine, it performs a chronological, walk-forward simulation that mimics real-world trading behavior. This includes applying user-defined rules for position sizing, order execution, slippage modeling, transaction costs, rebalancing schedules, stop-loss and trailing stop mechanisms, profit-taking rules, maximum portfolio drawdown protections, and other portfolio-level constraints such as sector caps or exposure limits. All simulations incorporate realistic delays between fund action dates and 13F publication dates to reflect actual information availability.

Crucially, this module must be designed as a decoupled micro-service exposing a strictly typed RESTful API. This API-first architecture is mandatory to ensure future extensibility, specifically to enable an LLM-based wrapper or automated agent to programmatically trigger multiple iterative backtests by dynamically varying input parameters (e.g., risk management thresholds, stop-loss percentages, entry / exit signals). This design allows for automated sensitivity analysis and hyperparameter optimization without requiring manual interaction through the UI.

{ Clarification needed: Please define the JSON schema for the Backtest Request and Response objects. A clear API contract (Swagger/OpenAPI) is required for the MVP to allow frontend and backend teams to work in parallel. }

Its role in the system is to translate theoretical signals into practical, time-consistent investment outcomes. It is responsible for constructing simulated trades, tracking positions across time, updating portfolio value, handling exits, and ensuring that the resulting performance metrics accurately reflect what would have happened in a real trading environment. If the Data Ingestion module provides the “facts” and the Signal Engine provides the “ideas,” then the Backtest Engine provides the “realism” needed for genuine evaluation.

The Backtest Engine outputs a complete set of artifacts: detailed trade logs, daily portfolio equity curves, position histories, risk metrics, per-trade R-multiples, exposure trajectories, and summary statistics. These outputs are consumed by the Analytics & Reporting Module for further visualization and interpretation. The Strategy Builder UI also relies on these outputs to display performance summaries and allow users to evaluate and iterate on their strategies.

Google Cloud Implementation:

In terms of cloud architecture, the Backtest Engine benefits from Cloud Run for containerized compute, or GKE for scalable, parallelized simulations. BigQuery stores backtest outputs and serves as the analytical layer for downstream reporting. Cloud Storage is used for exporting logs, and Pub/Sub enables parallel or distributed backtest workloads. Cloud Composer or Cloud

Run jobs can orchestrate complex workflows or batch processing of multiple strategies.

5.4 Strategy Builder UI Module

The Strategy Builder UI Module provides the user interface through which analysts, investors, or quant developers define and configure their custom investment strategies. It enables users to convert a base universe to sub-universes based on fund-level criteria (AUM thresholds, concentration rules, track records), trade-level criteria (new positions, increases, conviction levels), ticker level criteria, insider-buying filters, herding thresholds, fundamental, technical overlays, rebalancing frequencies, stop-loss and trailing stop rules, and overall portfolio constraints. All user selections are converted into a structured “strategy definition” that is passed to the backend in the form of a JSON configuration.

This module acts as the bridge between human intent and system logic. It ensures that users can create sophisticated strategies without writing code, and it validates the logical structure of rules before sending them to the Signal Engine and Backtest Engine. It is essentially the “authoring interface” that controls the entire strategy creation workflow.

The UI outputs the final strategy configuration and receives backtest results from downstream modules. It retrieves fund metadata, universe definitions, factor descriptions, and signal fields from the Data Ingestion module. It also consumes finished analytics (charts, tables, heatmaps) from the Analytics module and displays them in an interactive manner for the user.

Google Cloud Implementation:

On Google Cloud, the UI is best hosted via Firebase Hosting or Cloud Run. User strategies can be stored in Firestore for semi-structured JSON documents, or in BigQuery for structured analytical retrieval. Authentication is typically handled by Firebase Authentication. The UI communicates with backend services through API Gateway routes that forward requests to Cloud Run microservices. Exports and downloadable files are stored in Cloud Storage.

5.5 Analytics & Reporting Module

The Analytics & Reporting Module transforms raw backtest outputs and signal metadata into meaningful insights, performance evaluations, and visualizations. It computes all standard hedge-fund analytics including CAGR, volatility, Sharpe and Sortino ratios, maximum drawdowns, rolling performance windows, distribution of returns, sector-level performance decomposition, conviction-based attribution, factor-exposure measurements, and calendar-period heatmaps. It also generates visual charts such as equity curves, drawdown curves, bubble charts, radar charts, rolling Sharpe graphs, and annual return bars.

Within the system architecture, this module plays the role of the “interpreter.” It converts time-series data and trade logs into concise, decision-ready insights that help users understand the strengths, weaknesses, and behavioral patterns of their strategies. It allows for easy comparison between strategies and provides diagnostic tools that highlight which funds, sectors, or signals contribute meaningfully to performance.

The module consumes backtest output from the Backtest Engine, signal metadata from the

Signal Engine, and fundamentals and sector classification from the Data Ingestion module. It then produces charts, tables, performance summaries, attribution reports, and downloadable exports, all of which are delivered to the Strategy Builder UI for display to the end-user.

Google Cloud Implementation:

On Google Cloud, this module primarily works with BigQuery for analytical querying, BigQuery BI Engine for acceleration, and Cloud Run for server-side chart rendering or summary processing. Cloud Storage hosts static or exported chart assets, and Looker Studio may optionally be integrated for dashboard rendering. The module can also generate CSV or Excel exports that can be downloaded from Cloud Storage.

6. Appendix

6.1 Appendix A: Technical Architecture & Backtesting Library Recommendation

The user requires a Python library that satisfies three specific, non-negotiable constraints:

1. **Event-Driven:** (FR-3.1.D.1) mimicking real-time bar-by-bar processing.
2. **Fractional Shares:** (FR-3.1.D.4) allowing precise allocation of small capital.
3. **Custom Data:** (FR-3.1.B.2) ingesting non-price data (SEC 13F JSON) as a first-class citizen.

6.1.1 Landscape Analysis of Python Backtesters

The Python quantitative ecosystem is fragmented. We analyze the four primary contenders against the "Hedge Fund" persona requirements.

Feature	Backtrader	LEAN (QuantConnect)	Zipline-Reload ed	VectorBT
Architecture	Event-Driven (Loop)	Event-Driven (Institutional)	Event-Driven (Pipeline)	Vectorized (Array Ops)
Event-Driven Purity	High	Highest	High	Low (Simulated)
Fractional Shares	Hacky (Workarounds)	Native	Poor	Native
Custom Data	Excellent (GenericCSV)	Excellent (Data Sources)	Difficult (Ingest bundles)	Excellent
SEC Filing	Manual	Native Custom	Complex	Easy (Pandas)

Handling	Parsing required	Data Classes	Pipeline Factors	
Maintenance	Community Forks Only	Active (Commercial Team)	Community	Active
Learning Curve	Medium	High	High	Medium

6.1.2 The Recommendation: LEAN (via Skylight/Python)

Verdict: The **LEAN Engine** is the only candidate that robustly satisfies all "Hedge Fund" requirements without requiring "hacky" workarounds for fractional shares or custom data.

While **Backtrader** is popular for hobbyists, it lacks native fractional share logic (requiring users to rewrite the Broker class) and is effectively unmaintained. **VectorBT** is incredibly fast but acts as a vectorized simulator, not a true event-driven system (FR-3.1.D.1), making it risky for modeling complex order types and slippage.

Why LEAN Fits the SRS:

1. **Native Fractional Shares:** LEAN's transaction models handle decimal quantity precision natively.
 - *Configuration:* `self.Settings.AllowFractionalHoldings = True` satisfies FR-3.1.D.4 instantly.
2. **Custom Data Architecture for SEC Filings:**
 - LEAN allows the definition of a `PythonData` class. You can point this class to the BigQuery JSON export.
 - LEAN automatically synchronizes this data. If a 13F is timestamped 2023-11-14 16:00:00, LEAN feeds it to the `OnData()` method exactly then, preventing Look-Ahead Bias (FR-3.1.D.5).
3. **Event-Driven Loop:**
 - LEAN processes `OnData(slice)` where `slice` contains both the Price Bar (OHLC) and the SEC Filing Object. This allows the strategy to make atomic decisions: "I see the price is X and the Filing says Y, therefore Buy."
4. **Institutional Viability:**
 - Being backed by QuantConnect, it ensures that if the Hedge Fund decides to go live, the backtest code is deployable. Backtrader would require a complete rewrite for production.

6.2 Appendix B: Strategy Configuration UI Reference

Placeholder for "Sharpeley" strategy configuration screens.

6.3 Appendix C: Sample Backtest Report Visualizations

This section specifies the required visual output formats referenced in **FR-3.1.E.5**. The reporting module must generate these specific chart types to validate strategy robustness.

1. Monte Carlo Simulation: The Probability Cone

- **Purpose:** Visualizes the range of possible future equity curves based on randomized trade shuffling.
- **Components:**
 - **Blue Line:** The Original Backtest Equity Curve.
 - **Grey Lines:** 100+ Simulated Runs.
 - **Shaded Bands:** 95% and 99% Confidence Intervals.
- **Interpretation:** A narrow cone indicates a stable strategy; a wide cone implies high luck dependence.

2. Parameter Sensitivity: The Stability Heatmap

- **Purpose:** Tests if the strategy is overfitted to specific parameters (e.g., "Robustness Island").
- **Axes:**
 - **X-Axis:** Stop Loss Size (Points).
 - **Y-Axis:** Moving Average Period.
- **Color Scale:** Green (High Profit) to Red (Loss).
- **Goal:** Identify a "Green Island" (broad region of profitability) rather than a single green peak surrounded by red.

3. Walk Forward Testing: Out-of-Sample Cluster Matrix

- **Purpose:** Validates if the strategy maintains performance on unseen data.
- **Structure:** Grid of Re-optimization Period vs. Walk Forward Runs.
- **Metrics:** Walk Forward Efficiency (WFE) score per cell.
 - **Green:** WFE 0.6 - 1.2 (Good/Robust).
 - **Blue:** WFE > 1.2 (High Performance).
 - **Red:** WFE < 0.6 (Poor/Overfitted).

4. Stress Testing: Historical Crash Comparison

- **Purpose:** Benchmarks drawdown during known black swan events.
- **Components:** Side-by-side bar chart comparison.
 - **Blue Bar:** Strategy Max Drawdown %.
 - **Orange Bar:** Benchmark (S&P 500) Max Drawdown %.
- **Events:** Dot Com Bubble (2000), 2008 Financial Crisis, Covid Crash (2020), Inflation Spike (2022).